



Java String Methods

- charAt()
- codePointAt()
- codePointBefore()
- codePointCount()
- compareTo()
- compareToIgnoreCase()
- concat()
- contains()
- contentEquals()
- copyValueOf()
- endsWith()
- equals()
- equalsIgnoreCase()
- format()
- getBytes()
- getChars()
- hashCode()
- indexOf()
- isEmpty()
- join()
- lastIndexOf()
- length()
- matches()
- offsetByCodePoints()
- regionMatches()
- replace()
- replaceAll()
- replaceFirst()

Java String format() Method

◀ String Methods

Example

Return a formatted string:

```
String myStr = "Hello %s! One kilobyte is %,d bytes.";
String result = String.format(myStr, "World", 1024);
System.out.println(result);
```

Try it Yourself »

Note: You will find more "Try it Yourself" examples at the bottom of this page.

Definition and Usage

The `format()` method returns a formatted string using a locale, format and additional arguments.

If a locale is not passed to this method then the locale given by `Locale.getDefault()` is used.

Data from the additional arguments is formatted and written into placeholders in the format string, which are marked by a % symbol. The way in which arguments are formatted depends on the sequence of characters that follows the % symbol.

Placeholders

The placeholders have the form `%[arg$][flags][width][.precision]conversion` . The components in [square brackets] are optional.

An explanation of each of the components:

Get your own Java Server

ADVERTISEMENT

Get Certified!

Document your skills with W3Schools Certificates

\$1,995

\$699

Save 65%

🔒

COLOR PICKER



Java String Methods

- charAt()
- codePointAt()
- codePointBefore()
- codePointCount()
- compareTo()
- compareToIgnoreCase()
- concat()
- contains()
- contentEquals()
- copyValueOf()
- endsWith()
- equals()
- equalsIgnoreCase()
- format()
- getBytes()
- getChars()
- hashCode()
- indexOf()
- isEmpty()
- join()
- lastIndexOf()
- length()
- matches()
- offsetByCodePoints()
- regionMatches()
- replace()
- replaceAll()
- replaceFirst()

◦ `-` - Makes the output left-justified by adding any padding spaces to the right instead of to the left.

◦ `#` - Shows an alternate representation of the formatted data depending on the conversion.

◦ `+` - Causes positive numbers to always be prefixed with "+".

◦ - (A space character) This prefixes a space to positive numbers, primarily so that the digits can be lined up with the digits of negative numbers.

◦ `0` - Pads numbers with zeroes on the left.

◦ `,` - Groups digits (for example by thousands) and puts separators between the groups. This is affected by the locale.

◦ `(` - Encloses negative numbers in parentheses.

- `width` - Optional. A whole number specifying the minimum number of characters that the output should occupy. If necessary spaces are added to the right to reach this number, or to the left if the `-` flag is used.

- `.precision` Optional. A `.` followed by a whole number indicating how many decimal digits to show in the formatted data.

- `conversion` - Required. A character which indicates how an argument's data should be represented. If the character is uppercase the data will be formatted in uppercase where possible. The list of possible characters is shown in the table below.

List of conversions

Character	Conversion	Description
<code>%</code>	Percent	Displays a literal "%" character in the output.
<code>n</code>	Line break	Displays a line break in the output.
<code>b</code> or <code>B</code>	Boolean	Displays the boolean value of an argument as "true" or "false". If "B" is used then it displays "TRUE" or "FALSE" instead.
<code>h</code> or <code>H</code>	Unsigned hexadecimal integer	Represents an argument's binary data as an unsigned hexadecimal integer. If "H" is used then digits A to F are shown in uppercase. Note: For any data other than positive integers this does not represent its real value.
<code>s</code> or <code>S</code>	String	Displays the default string representation of the argument. If "S" is used then the string will be converted to uppercase where possible.
<code>c</code> or <code>C</code>	Unicode character	Displays a unicode character representation of the argument. For whole numbers, this is the unicode character that corresponds to the number. If "C" is used then the character will be converted to uppercase where possible.
<code>d</code>	Decimal integer	Represents a whole number as a decimal integer.
<code>o</code>	Octal integer	Represents a whole number as an octal integer. The "#" flag will prefix the number with "0".
<code>x</code> or <code>X</code>	Hexadecimal integer	Represents a whole number as a hexadecimal integer. The "#" flag will prefix the number with "0x". If "X" is used then digits A to F and the letter X are shown in uppercase.



Search for

REMOVE ADS

Java String Methods

- charAt()
- codePointAt()
- codePointBefore()
- codePointCount()
- compareTo()
- compareToIgnoreCase()
- concat()
- contains()
- contentEquals()
- copyValueOf()
- endsWith()
- equals()
- equalsIgnoreCase()
- format()
- getBytes()
- getChars()
- hashCode()
- indexOf()
- isEmpty()
- join()
- lastIndexOf()
- length()
- matches()
- offsetByCodePoints()
- regionMatches()
- replace()
- replaceAll()
- replaceFirst()

f	Floating point number	Represents a floating point number. The "#" flag will force a decimal point even if there are no decimal digits.
g or G	General number	Displays the shortest representation between f and e or E for a floating point number.
a or A	Hexadecimal floating point number	Display a floating point number's internal representation with hexadecimal digits.
t or T	Time or date	Displays a formatted date or time. The t or T must be followed by one more character indicating how the date or time should be formatted. If "T" is used then text parts of a date or time such as "JANUARY" will be uppercase. The following characters can be used for date and time formatting: - H - 24-hour format of an hour (00 to 23) - I - 12-hour format of an hour (01 to 12) - k - 24-hour format of an hour (0 to 23) - l (lowercase 'l') - 12-hour format of an hour (1 to 12) - M - Minutes with leading zeros (00 to 59) - S - Seconds with leading zeros (00 to 59) (The value 60 may occur for leap seconds) - L - Milliseconds with leading zeroes (000 to 999) - N - Nanoseconds with leading zeroes (000000000 to 999999999) - p - "am", "pm", "AM" or "PM" to indicate morning or afternoon - z - Difference to Greenwich time (Example: -0800) - Z - Timezone abbreviations (Examples: EST, MDT) - s - The seconds since the Unix Epoch (January 1 1970 00:00:00 GMT) - Q - The milliseconds since the Unix Epoch (January 1 1970 00:00:00 GMT) - B - A full textual representation of a month (January through December) - b or h - A short textual representation of a month (three letters) - A - A full textual representation of a day (Example: Monday) - a - A short textual representation of a day (Example: Mon) - C - The first two digits of the year (For 1970, "19" would be shown) - Y - A four digit representation of a year - y - A two digit representation of a year - j - The day of the year with leading zeroes (001 to 366) - m - A numeric representation of a month (01 to 12) - d - The day of the month (01 to 31) - e - The day of the month without leading zeros (1 to 31) - R - The time in 24-hour format (Example: 21:30) - T - The time in 24-hour format with seconds (Example: 21:30:02) - r - The time in 12-hour format with seconds (Example: 09:30:02 PM) ("AM" and "PM" are always uppercase) - D - Date representation as month/day/year (Example: 12/17/23)



Java String Methods ▾

- charAt()
- codePointAt()
- codePointBefore()
- codePointCount()
- compareTo()
- compareToIgnoreCase()
- concat()
- contains()
- contentEquals()
- copyValueOf()
- endsWith()
- equals()
- equalsIgnoreCase()
- format()
- getBytes()
- getChars()
- hashCode()
- indexOf()
- isEmpty()
- join()
- lastIndexOf()
- length()
- matches()
- offsetByCodePoints()
- regionMatches()
- replace()
- replaceAll()
- replaceFirst()

Syntax

One of the following:

```
public static String format(Locale locale, String format, Object... args)
```

```
public static String format(String format, Object... args)
```

Parameter Values

Parameter	Description
<i>locale</i>	Optional. A locale used to determine some of the formatting, such as which characters are used for decimal points and grouping separators.
<i>format</i>	Required. A string to be returned which can have placeholders for the additional arguments indicating how to format them.
<i>args</i>	Optional. Any number of additional arguments to the method, their values can be formatted and displayed in the returned string.

Technical Details

Returns:	A String value formatted using the specified locale, format and arguments.
Throws:	IllegalFormatException - if the format string contains an invalid placeholder or a placeholder isn't compatible with the data type of the argument.
Java version:	1.5

More Examples

Example

A placeholder which uses all of the components:



Java String Methods

- charAt()
- codePointAt()
- codePointBefore()
- codePointCount()
- compareTo()
- compareToIgnoreCase()
- concat()
- contains()
- contentEquals()
- copyValueOf()
- endsWith()
- equals()
- equalsIgnoreCase()
- format()
- getBytes()
- getChars()
- hashCode()
- indexOf()
- isEmpty()
- join()
- lastIndexOf()
- length()
- matches()
- offsetByCodePoints()
- regionMatches()
- replace()
- replaceAll()
- replaceFirst()

This is how each part of the placeholder `%2$,3.2f` works:

- `2$` indicates that the value of the second argument is used
- `,` indicates that digits should be grouped (usually by thousands)
- `3` indicates that the representation of the data should be at least 3 characters long
- `.2` indicates that there should be two digits after the decimal point
- `f` indicates that the data is being represented as a floating point number

Try it Yourself »

Example

Use arguments in a different order:

```
String result = String.format("%3$c %2$c %1$c", 'a', 'b', 'c');
System.out.println(result);
```

Try it Yourself »

Example

Format a floating point number:

```
double myNumber = 123456.78;
String result;

// Default
result = String.format("%f", myNumber);
System.out.println(result);

// Two decimal digits
result = String.format("%.2f", myNumber);
System.out.println(result);

// No decimal digits
result = String.format("%.0f", myNumber);
System.out.println(result);
```



Java String Methods ▼

- charAt()
- codePointAt()
- codePointBefore()
- codePointCount()
- compareTo()
- compareToIgnoreCase()
- concat()
- contains()
- contentEquals()
- copyValueOf()
- endsWith()
- equals()
- equalsIgnoreCase()
- format()
- getBytes()
- getChars()
- hashCode()
- indexOf()
- isEmpty()
- join()
- lastIndexOf()
- length()
- matches()
- offsetByCodePoints()
- regionMatches()
- replace()
- replaceAll()
- replaceFirst()

```
// Group digits
result = String.format("%.2f", myNumber);
System.out.println(result);

// Scientific notation with two digits of precision
result = String.format("%.2e", myNumber);
System.out.println(result);
```

Try it Yourself »

Example

Format a date from a Unix timestamp:

```
long date = 1711638903488L; // Unix timestamp (number of milliseconds since January 1, 1970)
String result

// Time
result = String.format("%t1:%<tM %<tp", date);
System.out.println(result);

// Month and day
result = String.format("%tB %<te", date);
System.out.println(result);

// Full date representation
result = String.format("%tc", date);
System.out.println(result);
```

Try it Yourself »

Example

Represent characters from their unicode code points:

Java String Methods

- charAt()
- codePointAt()
- codePointBefore()
- codePointCount()
- compareTo()
- compareToIgnoreCase()
- concat()
- contains()
- contentEquals()
- copyValueOf()
- endsWith()
- equals()
- equalsIgnoreCase()
- format()
- getBytes()
- getChars()
- hashCode()
- indexOf()
- isEmpty()
- join()
- lastIndexOf()
- length()
- matches()
- offsetByCodePoints()
- regionMatches()
- replace()
- replaceAll()
- replaceFirst()

```
result = String.format("%c%c%c%c%c", 72, 101, 108, 108, 111);
System.out.println(result);

// Force unicode characters to uppercase
result = String.format("%C%C%C%C%C", 72, 101, 108, 108, 111);
System.out.println(result);
```

Try it Yourself »

◀ String Methods

Sign in to track progress

ADVERTISEMENT

ADVERTISEMENT

Top Tutorials

- HTML Tutorial
- CSS Tutorial
- JavaScript Tutorial

Top References

- HTML Reference
- CSS Reference
- JavaScript Reference

Top Examples

- HTML Examples
- CSS Examples
- JavaScript Examples

Get Certified

- HTML Certificate
- CSS Certificate
- JavaScript Certificate



Java Tutorial
C++ Tutorial
jQuery Tutorial

Java Reference
AngularJS Reference
jQuery Reference

Java Examples
XML Examples
jQuery Examples

C++ Certificate
C# Certificate
XML Certificate



W3Schools is optimized for learning and training. Examples might be simplified to improve reading and learning. Tutorials, references, and examples are constantly reviewed to avoid errors, but we cannot warrant full correctness of all content. While using W3Schools, you agree to have read and accepted our [terms of use](#), [cookie and privacy policy](#).

Copyright 1999-2025 by Refsnes Data. All Rights Reserved. W3Schools is Powered by W3.CSS.

copyValueOf()
endsWith()
equals()
equalsIgnoreCase()
format()
getBytes()
getChars()
hashCode()
indexOf()
isEmpty()
join()
lastIndexOf()
length()
matches()
offsetByCodePoints()
regionMatches()
replace()
replaceAll()
replaceFirst()

